Région Hauts-de-France
Cofinancé par l'Union européenne
realconseil DIAGNOSTIC ACCOMPAGNEMENT FORMATION

PHP - Pack d'exercices (bases -> formulaires -> fichiers) - Niveau CDA

Règles générales (sauf mention contraire) :

- Travailles en PHP 8+.
- Affiches tes résultats en HTML (même simple).
- Utilises `htmlspecialchars(..., ENT_QUOTES, 'UTF-8')` pour toute donnée venant de l'utilisateur.
- Privilégies `===` et `!==` pour les comparaisons.
- Indentes proprement.

---

0 Mise en place (très court)

Exercice 0.1 - Hello + variables

- Crée `index.php`.
- Affiche : "Bonjour PHP".
- Déclare `$prenom`, `$age`, `$ville` et affiche une phrase complète avec ces variables.

Bonus :

- Affiche le type de chaque variable avec `var_dump`.

---

1 Conditions (niveau 1 -> 2)

Exercice 1.1 - Contrôle d'accès simple

Écris un script qui définit :

- `$age`
- `$aCarteIdentite` (bool)
- `$estAccompagne` (bool)

Règles :

- Si âge >= 18 : "Accès autorisé"
- Sinon si âge >= 16 ET accompagné : "Accès autorisé (accompagné)"
- Sinon : "Accès refusé"
- Si pas de carte d'identité : "Accès refusé (pièce manquante)" (prioritaire)

Bonus :

- Ajoute une variable `$heure` (0..23) et refuse l'accès après 22h.

Exercice 1.2 - Tarification (if / elseif)

Données :

- `$age`
- `$estEtudiant`
- `$estDemandeurEmploi`

Règles de prix :

- Enfant (< 12) : 4€
- Ado (12..17) : 6€
- Adulte : 10€
- Réductions (cumul interdit, prendre la meilleure) :
  - Étudiant : -20%
  - Demandeur d'emploi : -30%

Affiche :

- Prix de base
- Réduction appliquée
- Prix final (arrondi à 2 décimales)

Bonus :

- Si l'âge est invalide (<=0 ou >120), affiche une erreur.

Exercice 1.3 - Switch: jours ouvrés

Variable `$jour` (ex: "lundi", "mardi"... en minuscules).

- Affiche "ouvré" pour lundi..vendredi
- "week-end" pour samedi/dimanche
- Erreur si valeur inconnue

Bonus :

- Autorise aussi "Lundi" et "LUNDI" (normalisation avec `strtolower`).

---

2 Boucles (niveau 1 -> 2)

Exercice 2.1 - Table de multiplication

Variable `$n`.

- Affiche la table de `n` de 1 à 10 dans une liste HTML.

Bonus :

- Fais un tableau HTML (2 colonnes : "calcul" et "résultat").

Exercice 2.2 - Somme et moyenne

À partir d'un tableau de notes (ex: `[12, 9, 17, 14, 8]`) :

- Calcule la somme
- Calcule la moyenne
- Affiche la meilleure note, la pire note

Bonus :

- Compte le nombre de notes >= 10.

Exercice 2.3 - FizzBuzz (classique mais utile)

De 1 à 100 :

- "Fizz" si multiple de 3
- "Buzz" si multiple de 5
- "FizzBuzz" si multiple des deux
- Sinon le nombre

Bonus :

- Affiche dans un tableau HTML avec une colonne "type".

---

3 Tableaux (niveau 1 -> 3)

Exercice 3.1 - Panier (tableau associatif)

Crée un tableau associatif `$panier` :

```
$panier = [
  "pomme" => 1.2,
  "banane" => 0.9,
  "chocolat" => 2.5
];
```

- Affiche chaque produit + prix
- Calcule le total

Bonus :

- Ajoute un tableau `$quantites` et calcule un total avec quantités.

Exercice 3.2 - Catalogue (tableau multidimensionnel)

Crée un tableau `$produits` (au moins 6 produits) avec :

- `id` (int)
- `nom` (string)
- `prix` (float)
- `stock` (int)
- `categorie` (string)

Consignes :

- Affiche tous les produits sous forme de tableau HTML.
- Filtre et affiche seulement ceux en stock.
- Trie par prix croissant.

Bonus :

- Filtre par catégorie via une variable `$categorieChoisie`.

Exercice 3.3 - Statistiques par catégorie (niveau 3)

À partir de `$produits` :

- Calcule le nombre de produits par catégorie
- Calcule le prix moyen par catégorie
- Affiche un tableau récapitulatif

Indice :

- Utiliser un tableau `$stats` où la clé = catégorie.

---

4 Fonctions (niveau 1 -> 3)

Exercice 4.1 - prixTTC

Écrire :

```
function prixTTC(float $ht, float $taux = 0.2): float
```

- Retourne le TTC

Bonus :

- Ajoute une validation : si `$ht < 0` ou `$taux < 0`, lance une exception.

Exercice 4.2 - sanitize

Écrire :

```
function sanitize(string $s): string
```

- `trim`
- supprime les espaces multiples (remplacer par un seul)
- retourne la version nettoyée

Bonus :

- Ajoute une option pour mettre en "Title Case" (première lettre en majuscule).

Exercice 4.3 - Recherche dans un tableau

Écrire :

```
function findProductById(array $produits, int $id): ?array
```

- Retourne le produit trouvé, ou `null` sinon.

Bonus :

- Fais la même pour chercher par nom (insensible à la casse).

Exercice 4.4 - Génération HTML (séparation logique/affichage)

Écrire une fonction :

```
function renderTable(array $rows): string
```

- Retourne une string HTML d'un tableau.

Contraintes :

- La fonction ne doit pas faire de `echo`, seulement `return`.

Bonus :

- Gère le cas tableau vide (afficher "Aucune donnée").

---

5 Dates et heures (niveau 1 -> 2)

Exercice 5.1 - Date du jour + formats

- Affiche la date du jour au format `d/m/Y`.
- Affiche aussi `Y-m-d H:i:s`.

Bonus :

- Affiche le jour de la semaine en français (au moins via une table de correspondance).

Exercice 5.2 - Calcul d'échéance

Tu as :

- une date de départ (string `YYYY-MM-DD`)
- un nombre de jours (int)

Consignes :

- Calculer la date d'échéance et l'afficher.
- Dire si l'échéance est dans le passé, aujourd'hui, ou dans le futur.

Bonus :

- Ajoute un cas "ouvré" : si l'échéance tombe samedi/dimanche, reporte au lundi.

Exercice 5.3 - Âge exact (niveau 2)

À partir d'une date de naissance `YYYY-MM-DD` :

- Calculer l'âge en années
- Afficher aussi le nombre de jours avant le prochain anniversaire

---

6 Formulaires et superglobales (niveau 1 -> 3)

Exercice 6.1 - Formulaire "profil"

Créer `profil.php` :

- Formulaire POST : prénom, nom, email, ville, âge
- À la soumission :
  - valider champs obligatoires
  - âge doit être un entier 1..120
  - email doit contenir `@` (simple)
- Afficher un récapitulatif propre en HTML

Bonus :

- Pré-remplir les champs avec les valeurs soumises (confort UX).

Exercice 6.2 - Calculatrice (niveau 2)

Formulaire POST :

- a (nombre)
- b (nombre)
- opération (select : +, -, *, /)

Consignes :

- Valider les nombres
- Empêcher division par 0
- Afficher le résultat

Bonus :

- Conserver l'opération choisie après soumission.

Exercice 6.3 - Recherche en GET (niveau 2)

Créer `recherche.php` :

- Un champ GET `q`
- Un tableau de données en dur (ex: liste de villes ou produits)
- Filtrer ce tableau pour afficher seulement les éléments contenant `q` (insensible à la casse)

Bonus :

- Afficher le nombre de résultats.

Exercice 6.4 - Mini-auth (niveau 3)

Créer `login.php` :

- Formulaire POST : user + password
- Identifiants attendus en dur : `admin / admin123`
- Si OK : afficher "Connecté"
- Sinon : "Identifiants invalides"

Bonus :

- Ajoute un compteur de tentatives (stocké en session).

---

7 Fichiers (niveau 2 -> 3)

Exercice 7.1 - Compteur de visites

Créer `compteur.php` et un fichier `data/compteur.txt` :

- Si fichier absent, initialise à 0
- À chaque chargement, +1
- Affiche la valeur

Bonus :

- Empêche les valeurs non numériques (si le fichier est corrompu, remettre à 0).

Exercice 7.2 - Livre d'or (niveau 3)

Créer `livre_or.php` :

- Formulaire POST : pseudo + message
- Enregistrer dans `data/messages.txt` au format : `YYYY-MM-DD HH:MM|pseudo|message`

Affichage :

- Lire le fichier et afficher les 20 derniers messages (ordre du plus récent au plus ancien)
- Échapper les données à l'affichage

Bonus :

- Refuser message > 200 caractères
- Filtrer par pseudo via GET `?pseudo=...`

Exercice 7.3 - Export CSV (niveau 3)

À partir de ton catalogue `$produits` :

- Générer un fichier `data/produits.csv`
- En-tête : `id;nom;prix;stock;categorie`
- Une ligne par produit

Bonus :

- Ajouter une page `telecharger.php` qui force le téléchargement du CSV (headers HTTP).

---

8 Upload de fichiers (niveau 3, sécurité)

Exercice 8.1 - Upload contrôlé

Créer `upload.php` :

- Formulaire d'upload (multipart)
- Autoriser uniquement : jpg, png, pdf
- Taille max : 2 Mo
- Renommer côté serveur (nom aléatoire)
- Stocker dans `data/uploads/`

Bonus sécurité :

- Interdire tout fichier php
- Ne jamais stocker dans un dossier exécutable par le serveur (si possible)

---

9 Mini-projet (2 à 4 heures)

Projet - "Gestion de stock simplifiée"

But : une mini-app sans base de données (fichier JSON ou CSV)

Fonctionnalités minimum :

- Page liste produits (tableau HTML)
- Formulaire ajout produit (nom, prix, stock, catégorie)
- Enregistrement dans `data/produits.json`
- Validation (prix >= 0, stock >= 0, nom non vide)
- Possibilité de supprimer un produit (par id)

Bonus :

- Filtre par catégorie (GET)
- Tri par prix
- Page détail produit
- Export CSV

Contraintes :

- Utiliser des fonctions : lecture JSON, écriture JSON, validation, rendu HTML
- Échapper toute donnée utilisateur